Exception Handling Experiments:

1.
```
public class ZeroDivError{
    public static void main(String[] args){
        try{
            int c = 10 / 0;
            System.out.println("Result:"+c);
        }
        catch (ArthmeticException e){
            System.out.println("Error: Division by zero");
        }
        System.out.println("Code executed");
    }
}
```

Output:
ERROR!
Error: Division by zero
Code executed

2.
```
public class Exceptiontypes {
    public static void main(String[] args) {
        try {
            int a = 5 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Caught ArithmeticException: " + e.getMessage());
        }

        try {
            int[] arr = new int[3];
            int val = arr[5];
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Caught ArrayIndexOutOfBoundsException: " +
e.getMessage());
        }

        try {
            String str = null;
            int length = str.length();
        } catch (NullPointerException e) {
            System.out.println("Caught NullPointerException: " + e.getMessage());
        }
    }
}
```

Output:
Caught ArithmeticException: / by zero
Caught ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
Caught NullPointerException: Cannot invoke "String.length()" because "<local1>" is null

3.
```
public class UncaughtExceptionDemo {
    public static void main(String[] args) {
        int num = 50 / 0; // Uncaught exception, terminates program and prints stack trace
        System.out.println("This line will not execute.");
    }
}
```

Output:
ERROR!
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at Main.main(Main.java:6)

4.
```
public class MultipleCatch{
    public static void main(String[] args) {
        try {
            String s = null;
            System.out.println(s.length()); // Triggers NullPointerException
            int x = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic exception handled.");
        } catch (NullPointerException e) {
            System.out.println("Null pointer exception handled.");
        } catch (Exception e) {
            System.out.println("General exception handled.");
        }
    }
}
```

Output:
Null pointer exception handled.

5.
```
public class Throw{
    public static void checkAge(int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Age must be 18 or older.");
        } else {
            System.out.println("Access granted.");
```

```
        }
    }
    public static void main(String[] args) {
        try {
            checkAge(15);
        } catch (IllegalArgumentException e) {
            System.out.println("Caught explicit exception: " + e.getMessage());
        }
    }
}
```

Output:
Caught explicit exception: Age must be 18 or older.

```
6. import java.io.IOException;

public class Throws{
    static void validateFile() throws IOException {
        throw new IOException("Device error/File error.");
    }

    public static void main(String[] args) {
        try {
            validateFile();
        } catch (IOException e) {
            System.out.println("Caught propagated exception: " + e.getMessage());
        }
    }
}
```

Output:
Caught propagated exception: Device error/File error.

```
7. public class Finally{
    public static void main(String[] args) {
        try {
            int data = 25 / 5;
            System.out.println(data);
        } catch (ArithmeticException e) {
            System.out.println(e);
        } finally {
            System.out.println("Finally block is always executed.");
        }
    }
```

```
}
```

Output:
5
Finally block is always executed.

8.
```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class UserDefinedException {
    public static void main(String[] args) {
        int age = 12;
        try {
            if (age < 18) {
                throw new InvalidAgeException("Not eligible to vote.");
            }
        } catch (InvalidAgeException e) {
            System.out.println("Custom Exception: " + e.getMessage());
        }
    }
}
```

Output:
Custom Exception: Not eligible to vote.

9.
```
class InvalidMarksException extends Exception {
    public InvalidMarksException(String msg) {
        super(msg);
    }
}
public class MarksValidation {
    public static void validateMarks(int marks) throws InvalidMarksException {
        if (marks < 0 || marks > 100) {
            throw new InvalidMarksException("Marks must be between 0 and 100.");
        }
        System.out.println("Marks are valid: " + marks);
    }

    public static void main(String[] args) {
        try {
            validateMarks(105);
```

```
        } catch (InvalidMarksException e) {
            System.out.println("Validation Error: " + e.getMessage());
        }
    }
}
```

Output:
Validation Error: Marks must be between 0 and 100.

10.
```
class OutOfRangeEx extends Exception {
    OutOfRangeEx(String s) { super(s); }
}

public class AllExceptions {
    static void checkValue(int val) throws OutOfRangeEx {
        if (val > 50) {
            throw new OutOfRangeEx("Value exceeds limit 50.");
        }
        System.out.println("Valid value.");
    }

    public static void main(String[] args) {
        try {
            int num = 10;
            int res = num / 2;
            checkValue(60); // Triggers custom exception
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic caught.");
        } catch (OutOfRangeEx e) {
            System.out.println("Custom caught: " + e.getMessage());
        } finally {
            System.out.println("Finally block executed cleanly.");
        }
    }
}
```

Output:
Custom caught: Value exceeds limit 50.
Finally block executed cleanly.

I/O streams: The Byte Streams Experiments:

1.
```java
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.IOException;

public class ByteStream {
    public static void main(String[] args) throws IOException {
        byte[] data = {65, 66, 67, 68}; // ASCII for A, B, C, D
        ByteArrayInputStream input = new ByteArrayInputStream(data);
        ByteArrayOutputStream output = new ByteArrayOutputStream();

        int val;
        while ((val = input.read()) != -1) {
            output.write(val);
        }

        System.out.println("Output bytes as string: " + output.toString());
    }
}
```

Output:
Output bytes as string: ABCD

2.
```java
import java.io.FileInputStream;
import java.io.IOException;

public class ReadFileByteByByte {
    public static void main(String[] args) {
        try (FileInputStream fis = new FileInputStream("test.txt")) {
            int i;
            while ((i = fis.read()) != -1) {
                System.out.print((char) i);
            }
        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        }
    }
}
```

Output:
Error reading file: test.txt (No such file or directory)

3.
```java
import java.io.FileOutputStream;
import java.io.IOException;

public class WriteFileByte {
    public static void main(String[] args) {
        String text = "Learning Byte Streams in Java.";
        try (FileOutputStream fos = new FileOutputStream("output.txt")) {
            fos.write(text.getBytes());
            System.out.println("Data successfully written to output.txt");
        } catch (IOException e) {
            System.out.println("Error writing file: " + e.getMessage());
        }
    }
}
```

Output:
Error writing file: output.txt (Permission denied)

4.
```java
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class CopyFileBytes {
    public static void main(String[] args) {
        try (FileInputStream fis = new FileInputStream("source.txt");
             FileOutputStream fos = new FileOutputStream("destination.txt")) {
            int temp;
            while ((temp = fis.read()) != -1) {
                fos.write(temp);
            }
            System.out.println("File copied successfully.");
        } catch (IOException e) {
            System.out.println("I/O Error: " + e.getMessage());
        }
    }
}
```

Output:
ERROR!
I/O Error: source.txt (No such file or directory)

5.
```java
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.FileNotFoundException;
```

```java
import java.io.IOException;

public class CopyImageBytes {
    public static void main(String[] args) {
        FileInputStream fis = null;
        FileOutputStream fos = null;
        try {
            fis = new FileInputStream("source_image.jpg");
            fos = new FileOutputStream("destination_image.jpg");

            byte[] buffer = new byte[1024];
            int length;
            while ((length = fis.read(buffer)) > 0) {
                fos.write(buffer, 0, length);
            }
            System.out.println("Image copied successfully using try-catch-
finally.");
        } catch (FileNotFoundException e) {
            System.out.println("File not found: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("I/O exception occurred: " + e.getMessage());
        } finally {
            try {
                if (fis != null) fis.close();
                if (fos != null) fos.close();
            } catch (IOException e) {
                System.out.println("Error closing streams: " + e.getMessage());
            }
        }
    }
}
```

Output:

File not found: source_image.jpg (No such file or directory)